# 第一个LangChain应用

- 来源: https://class.imooc.com/lesson/5283#mid=105031
- 阶段: 阶段四: 【框架应用】高效AI开发
- 周: 第8周 LangChain框架入门
- 模块: RAG集成：打造智能知识库
- 课节: 图文 1-1 第一个LangChain应用

## 1-1 第一个LangChain应用

## 第一个LangChain应用

> [!TIP] 阅读指南 本节会以“Level 1 ~ Level 5”的形式，逐步把一个简单的模型调用，升级成带 Prompt、带记忆、可交互的原神攻略助手。 建议先快速通读一遍，弄清每个 Level 的“新能力”是什么，然后再回头对照代码动手运行。 不需要死记硬背代码细节，更重要的是理解：什么时候只用模型，什么时候要加 Prompt，什么时候需要 Chain 和 Memory 来帮你管理流程和上下文。

## 应用目标

目标: 一个会聊天、能记忆、可查询的原神攻略助手

目标:

技术栈:

- LangChain
- Qwen 大模型
- 对话历史管理

这就像搭积木:先搭一个基础版本，然后逐步增强。

## 环境准备

安装依赖:

```
pip install "langchain==1.2.15" "langchain-openai==1.2.1"

1
```

langchain 是主框架包，会自动安装 langchain-core、langchain-text-splitters 等子模块。
langchain-openai 是 OpenAI 兼容接口的适配器（通义千问也兼容 OpenAI 协议，所以直接用这个）。

> [!TIP] 完整的源码目录结构与运行指引，请参考 samples/chapter8/langchain_demo/README.md。

为什么装 langchain-openai?

LangChain 的模型集成是模块化的:

- langchain - LangChain 的基础包
- langchain-core - LangChain 的核心依赖
- langchain-openai - 兼容 OpenAI 接口(由于 Qwen 也兼容 OpenAI,所以 Qwen 可以直接用这个)

配置 API Key:

```
import os

# Qwen API 配置
os.environ["OPENAI_API_KEY"] = "your-qwen-api-key"
os.environ["OPENAI_API_BASE"] = "https://dashscope.aliyuncs.com/compatible-mode/v1"
```

> 注意，这里我们将Qwen的API_KEY和地址写入到OPENAI的相关环境变量。LangChain在运行的过程中，会自动寻找这些环境变量，不需要我们在手动赋值了。

## Level 1: 最简单的对话

从最基础的开始 - 调用一次大模型。

> [!TIP] 完整源码参考：samples/chapter8/langchain_demo/01_basic_call.py

```
from langchain_openai import ChatOpenAI

# 初始化模型
llm = ChatOpenAI(
    model="qwen3.6-plus"
    temperature=0.7,
)

# 调用
response = llm.invoke("原神好玩吗?")
print(response.content)
```

运行效果:

```
《原神》是一款由米哈游（miHoYo）开发的开放世界冒险游戏，自2020年发布以来受到了全球玩家的喜爱。.....

1
```

这里发生了什么?

- ChatOpenAI 封装了 API 调用细节
- invoke() 方法发送请求

```
invoke()
```

- 返回 AIMessage 对象，.content 是回复文本

```
AIMessage
```

但这还不算 LangChain 应用，只是用了它的模型封装。真正的价值在后面。

## Level 2: 使用 Prompt 模板

手写 Prompt 太麻烦了，用模板管理更优雅。

[!TIP] 完整源码参考：samples/chapter8/langchain_demo/02_with_prompt.py

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

# 定义角色与任务
system_prompt = """你是一位资深的原神游戏攻略专家。

你的特点:
- 熟悉所有角色、武器、圣遗物配置
- 提供实用的配队建议和养成攻略
- 语言简洁,重点突出

请根据用户问题提供专业建议。"""

# 创建 Prompt 模板
prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    ("user", "{question}")
])

# 初始化模型
llm = ChatOpenAI(
    model="qwen3.6-plus"
    temperature=0.7,
)

# 组装成 Chain
chain = prompt | llm

# 使用
response = chain.invoke({"question": "雷电将军怎么配队?"})
print(response.content)
```

```
雷电将军配队核心思路：**充能+后台输出+元素反应/辅助**。以下是主流配队推荐：
.........

1
2
```

Level 2 相比 Level 1 的三大进步：

① 引入 Prompt 模板 - 管理复杂提示词

Level 1 的问题：

```
# 简单问题还好
response = llm.invoke("原神好玩吗?")

# 但如果需要设定角色和规则就比较麻烦
response = llm.invoke([
    {"role": "system", "content": "你是一位资深的原神游戏攻略专家。你的特点：熟悉所有角色..."},
    {"role": "user", "content": "雷电将军怎么配队?"}
])
```

Level 2 的解决方案：

```
# 定义一次模板
prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt),  # system_prompt 定义了AI的角色和规则
    ("user", "{question}")       # {question} 是占位符，使用时填入具体问题
])

# 之后只需传入业务参数
chain.invoke({"question": "雷电将军怎么配队?"})
```

ChatPromptTemplate把复杂的消息结构封装成可复用的模板：

```
# 定义角色与规则（写一次，到处用）
system_prompt = """你是一位资深的原神游戏攻略专家。

你的特点:
- 熟悉所有角色、武器、圣遗物配置
- 提供实用的配队建议和养成攻略
- 语言简洁,重点突出

请根据用户问题提供专业建议。"""

# 创建模板（system 消息 + user 消息）
prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt),  # 固定的角色设定
    ("user", "{question}")      # 动态的用户问题
])
```

② 管道语法 - 串联组件流程

还记得第 2 节中提到的管道“全自动流水线”吗？在 Level 1 中，我们需要手动获取响应并打印。到了 Level 2，通过 | 符号，我们将 Prompt 和模型“焊接”在了一起。

```
# ✓ 管道写法（声明式流程，数据自动流转）
chain = prompt | llm
response = chain.invoke({"question": "..."})
```

小结：Level 2 的本质 Level 1 是"直接调用"，Level 2 引入了组件化思维：

- Prompt 模板 = 可复用的提示词管理
- 管道语法 = 声明式的流程编排 这就是 LangChain 的设计哲学：用组件组合替代代码拼接。

## Level 3: 增加对话记忆

我们之前讨论过，AI并不记得之前说过什么，因为AI没有记忆。我们之前都是手动把之前的会话内容携带在提示词里。那么来看看LangChain怎么优雅的解决这个问题：

> [!TIP] 完整源码参考：samples/chapter8/langchain_demo/03_with_memory.py

测试一下:

```
# 第一轮
response1 = chain.invoke({"question": "我想玩雷电将军"})

# 第二轮
response2 = chain.invoke({"question": "她怎么配队?"})
# AI 不知道"她"指的是雷电将军
```

解决方案 - 添加 Memory:

要让 AI 记住对话，在 LangChain 里可以先记住一个简单的“三步法”：

- 在提示词里留出一个位置,专门放「之前说过的话」。

留出一个位置

- 准备一个地方专门存这次对话的来往消息(就像聊天记录)。
- 用一个**“记忆包装器”**把原来的对话链包起来，让它在每次调用前自动把历史塞进去，调用后再把这次对话补充到历史里。

下面的代码就是按照这三步展开的实现，代码里出现的那些类名，你可以先把它们当成完成这三步的“工具箱”。其中:

- InMemoryChatMessageHistory 可以理解为「放在内存里的聊天记录本」;
- RunnableWithMessageHistory 可以理解为前面提到的「记忆包装器」。

```
# 系统提示词
system_prompt = """你是一位资深的原神游戏攻略专家。
请根据对话历史和用户问题提供专业建议。"""

# Prompt 模板(增加历史占位符)
# 第一步：留出位置
prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    MessagesPlaceholder(variable_name="history"),  # 专门放「之前说过的话」的位置
    ("user", "{question}")
])

# 初始化模型
llm = ChatOpenAI(
    model="qwen3.6-plus"
    temperature=0.7,
)

# 基础 Chain
base_chain = prompt | llm

# 第二步：准备存储聊天记录的地方
# 用一个字典保存这次对话的所有消息,键是 session_id
chat_histories = {}  # 比如这次我们会用 'chat_001' 作为对话的名字

def get_chat_history(session_id: str):
    """根据 session_id 获取对应对话的聊天记录

    session_id 就是给这次对话起的名字,比如 "chat_001"
    """
    if session_id not in chat_histories:
        # 如果这是个新对话，创建一个新的历史记录
        chat_histories[session_id] = InMemoryChatMessageHistory()
    return chat_histories[session_id]

# 第三步：用“记忆包装器”包起来
# 这个包装器会自动做两件事：
# 1. 对话前：把历史记录拿出来，填到 "history" 位置
# 2. 对话后：把这次的问题和AI回答存起来
chain_with_history = RunnableWithMessageHistory(
    base_chain,                         # 原本的Chain
    get_chat_history,                   # 怎么获取历史的函数
    input_messages_key="question",    # 用户问题的字段名
```

```
    history_messages_key="history",   # 历史消息的字段名
)

# 使用：需要指定对话标识(session_id)
config = {"configurable": {"session_id": "chat_001"}}  # 给这次对话起个名字

# 第一轮对话
response1 = chain_with_history.invoke(
    {"question": "我想玩雷电将军"},
    config=config  # 携带对话标识
)
print("回复1:", response1.content)

# 第二轮对话（用同一个 session_id，所以AI能记得上次对话）
response2 = chain_with_history.invoke(
    {"question": "她怎么配队?"},  # AI知道"她"指的是雷电将军
    config=config  # 同一个 session_id
)
print("回复2:", response2.content)
```

```
回复1: 雷电将军是非常强力的充能辅助,可以...

回复2: 雷电将军的配队推荐如下:
       (AI 知道"她"指的是雷电将军!)
       ...
```

## Level 4: 输出结构化

有时候需要模型返回 JSON 格式的结构化数据。LangChain 提供了与 Pydantic 结合的方案，只需定义一个数据类和一个解析器：

```
from pydantic import BaseModel, Field
from langchain.output_parsers import PydanticOutputParser

# 定义数据结构
class TeamRecommendation(BaseModel):
    team_name: str = Field(description="队伍名称")
    members: list[str] = Field(description="队伍成员列表")

# 创建解析器
parser = PydanticOutputParser(pydantic_object=TeamRecommendation)

# 组装链条
chain = prompt | llm | parser

# 调用后直接拿到结构化对象
result = chain.invoke({
    "character": "雷电将军",
    "format_instructions": parser.get_format_instructions()
})
print(result.team_name)  # 直接访问字段
```

## Level 5: 流式输出

用户不想等待完整响应，希望看到 AI 逐字输出。LangChain 支持流式调用：

```
# 用 stream() 替代 invoke() 即可
for chunk in chain.stream({"question": "雷电将军的培养优先级"}):
    print(chunk.content, end="", flush=True)
```

invoke() 等待完整响应后一次性返回，stream() 逐块返回、实时显示。使用方式完全一致，只需把 .invoke() 换成 .stream()。

## 完整示例

以上是 LangChain 的基础用法。完整的交互式对话示例（整合记忆+流式输出）请参考源码：

> [!TIP] 完整源码参考：samples/chapter8/langchain_demo/06_complete_chat.py

## ■ 学点英语

| 中文 | English | 音标 | 说明 |
| --- | --- | --- | --- |
| 聊天提示模板 | ChatPromptTemplate | /tʃæt prɑːmpt ˈtemplət/ | 管理 system/user 消息结构的模板类 |
| 消息占位符 | MessagesPlaceholder | /ˈmesɪdʒɪz ˈpleɪshoʊldər/ | 在模板中预留历史对话的插入位置 |
| 会话标识 | Session ID | /ˈseʃn aɪ diː/ | 区分不同对话会话的唯一标识符 |
| 流式输出 | Stream Output | /striːm ˈaʊtpʊt/ | 逐块返回响应而非一次性完成的模式 |

# 下一节预告

无论是 Prompt 模板还是对话记忆，AI 始终只能"说话"，无法真正"行动"。如果想让 AI 查询实时天气、搜索最新攻略或者操作数据库，就需要用到 LangChain 的工具调用机制。这是下一节将要讨论的内容。